

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	G. Nithya Sree	Reg. No.:	192424103
Section / Batch:	192424103 3rd year, AID	Date:	02/09/2026

Code of Conduct

I, G. Nithya Sree (192424103) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G. Nithya Sree

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge

Course Coordinator

Program Director

Signature: _____

Signature: _____

Signature: _____

Date: _____

Date: _____

Date: _____

Breadth First Search (BFS):-

Aim:- To perform Breadth first search traversal of a graph using a queue.

Algorithm:-

- 1) Read the graph and source vertex.
- 2) Mark all vertices as unvisited.
- 3) Insert ^{the} source vertex into the queue and mark it visited.
- 4) Delete a vertex from the queue and print it.
- 5) Visit all its unvisited neighbours and insert them into the queue.
- 6) Repeat until the queue becomes empty.

C-program:-

```
#include <stdio.h>
```

```
#define MAX 20
```

```
int main () {
```

```
    int a [MAX][MAX], visited [MAX], = {0};
```

```
    int q [MAX], front = 0, rear = 0;
```

```
    int n, e, u, v, s, i, x;
```

```
    printf("Enter vertices and edges:");
```

```
    scanf("%d %d", &n, &e);
```

```
    for(i = 0; i < e; i++) {
```

```
        scanf("%d %d", &u, &v);
```

```
        a[u][v] = a[v][u] = 1;
```

```
    }
```

```
    printf("Enter source:");
```

```
    scanf("%d", &s);
```

```
    q[rear++] = s;
```

```
    visited[s] = 1;
```

```
printf("BFS: ");
```

```
while (front < rear) {
```

```
    x = q[front++];
```

```
    printf("%d ", x);
```

```
    for (i = 0; i < n; i++) {
```

```
        if (a[x][i] && !visited[i]) {
```

```
            visited[i] = 1;
```

```
            q[rear++] = i;
```

```
        }
```

```
    }
```

```
    return 0;
```

Sample input:-

6 5

0 1

0 2

0 3

3 4

4 5

0

Sample output:-

BFS: 0 1 2 3 4 5

2) Kruskal's Algorithm:-

Aim:- To find the minimum spanning tree of a weighted graph using Kruskal's Algorithm.

Algorithm:-

- 1) Read the weighted graph.
- 2) Sort all edges in increasing order of weight.
- 3) Initially put every vertex in a separate set.
- 4) Select the smallest edge

If it does not form a cycle, add it to the MST.
Repeat until $V-1$ edges are selected.

C-Program:-

```
#include <stdio.h>
```

```
struct e {  
    int u, v, w;
```

```
};
```

```
int p[10];
```

```
int find(int x) {
```

```
    while (p[x] != x)
```

```
        x = p[x];
```

```
    return x;
```

```
}
```

```
int main () {
```

```
    struct e e[20], t;
```

```
    int n, m, i, j, a, b;
```

```
    int count = 0, cost = 0;
```

```
    scanf("%d %d", &n, &m);
```

```
    for (i = 0; i < m; i++)
```

```
        scanf("%d %d %d", &e[i].u, &e[i].v, &e[i].w);
```

```
    for (i = 0; i < m-1; i++)
```

```
        for (j = i+1; j < m; j++)
```

```
            if (e[i].w > e[j].w)
```

```
            {
```

```
                t = e[i];
```

```
                e[i] = e[j];
```

```
                e[j] = t;
```

```
            }
```

sample input:

4 5

0 1 10

0 2 6

0 3 5

1 3 15

2 3 4

Output:

MST:

2-3 = 4

0-3 = 5

0-1 = 10.

Cost = 19.

```
printf("MST:\n");
```

```
for (i = 0; i < m && count < n-1; i++) {
```

```
    a = find(e[i].u);
```

```
    b = find(e[i].v);
```

```
    if (a != b) {
```

```
        printf("%d - %d = %d\n", e[i].u, e[i].v, e[i].w);
```

```
        p[a] = b;
```

```
        cost += e[i].w;
```

```
        count++;
```

```
    }
```

```
    printf("cost = %d", cost);
```

```
    return 0;
```

```
}
```

3) Topological sort:-

Aim: To find the topological ordering of a directed acyclic graph using Khan's Algorithm.

Algorithm:-

- 1) Read the directed graph.
- 2) Find the indegree of every vertex.
- 3) Insert all vertices with indegree 0 into the queue.
- 4) Decrease the indegree of its neighbours.
- 5) Continue until the queue is empty.

Code:-

```
#include <stdio.h>
```

```
int main () {
```

```
int a[10][10] = {0}, in[10] = {0}, q[10],
```

```
int n, e, i, x, y, f = 0, r = 0;
```

```
scanf("%d %d", &n, &e);
```

```
for (i = 0; i < e; i++) {
```

```
    scanf("%d %d", &x, &y);
```

```
    a[x][y] = 1;
```

```
    in[y]++;
```

```
}
```

```
for (i = 0; i < n; i++)
```

```
    if (in[i] == 0)
```

```
        q[r++] = i;
```

```
while (f < r)
```

```
{
```

```
    x = q[f++];
```

```
    printf("%d", x);
```

```
    for (i = 0; i < n; i++)
```

```
        if (a[x][i] && in[i] == 0)
```

```
            q[r++] = i;
```

```
}
```

```
return 0;
```

```
}
```

Sample input:-

6 6

5 2

5 0

4 0

4 1

2 3

3 1

Output: 4 5 2 0 3 1

4) DFS Tree:-

Aim:- To generate a Depth first search tree using recursive DFS.

Algorithm:-

- 1) Read the graph and source vertex.
- 2) Mark the source as visited.
- 3) Explore its neighbour one by one.
- 4) Recursively perform DFS on that neighbour.

5) continue untill all reachable vertices are visited.

C-code:-

```
#include <stdio.h>
int a[10][10], v[10], n;
void DFS (int x) {
    int i;
    v[x] = 1;
    printf("%d ", x);
    for (i = 0; i < n; i++) {
        if (a[x][i] && !v[i])
            DFS (i);
    }
}

int main () {
    int e, x, y, s, i;
    scanf ("%d %d", &n, &e);
    for (i = 0; i < e; i++) {
        scanf ("%d %d", &x, &y);
        a[x][y] = a[y][x] = 1;
    }
    scanf ("%d", &s);
    printf ("DFS: ");
    DFS (s);
    return 0;
}
```

sample input:-

```
6 5
0 1
0 2
0 3
3 4
4 5
0 .
```

output:-

DFS : 0 1 2 3 4 5

Prim's Algorithm:-

Aim:- To find the minimum spanning tree of a weighted graph using prim's algorithm.

Algorithm:-

- 1) Read the weighted graph.
- 2) Start from a selected vertex.
- 3) Find the minimum-weight edge from a visited vertex to an unvisited vertex.
- 4) Mark the new vertex as visited.
- 5) Repeat until all vertices are included.

C-Code:-

```
#include <stdio.h>

int main () {

    int a[10][10] = {0}, v[10] = {0};

    int n, e, i, x, y, w, min, u, z, cost = 0;

    scanf("%d %d", &n, &e);

    for (i = 0; i < e; i++) {

        scanf("%d %d %d", &x, &y, &w);

        a[x][y] = a[y][x] = w;

    }

    v[0] = 1;

    for (i = 1; i < n; i++) {

        min = 999;

        for (x = 0; x < n; x++)

            if (v[x])
```

```
for (y = 0; y < n; y++)
```

```
if (!v[y] && a[x][y] && a[x][y] < min) {
```

```
    min = a[x][y];
```

```
    u = x;
```

```
    z = y;
```

```
}
```

```
printf("u = %d, z = %d\n", u, z, min);
```

```
cost += min;
```

```
v[z] = 1;
```

```
}
```

```
printf("cost = %d", cost);
```

```
return 0;
```

y.

Sample input:

5 7

0 1 2

0 3 6

1 2 3

1 3 8

1 4 5

2 4 3

3 4 9.

output:

0 - 1 = 2.

1 - 2 = 3

1 - 4 = 5

0 - 3 = 6.

cost = 16.